Durham University

Applied security (COMP3761)
IoT Hacking

Prof David Oswald & Dr Charles Morisset

david.f.oswald@durham.ac.uk

MCS1033

Plan for today

After this lecture, you will …

- have an understanding what firmware extraction is and why it matters
- know which different techniques exist
- understand the relative difficulty of each method
- realize that firmware can never be 100% protected



Computational capabilities / power budget

What is firmware?

- **Wikipedia:** “In computing, firmware is a specific class of computer software that provides the low-level control for a device's specific hardware.”
- Firmware = OS and application logic for embedded devices (sometimes without OS as “bare metal”)
- Virtually all modern embedded devices are programmable with firmware (and not hardwired)
- Embedded systems can contain multiple processors with their own firmware binary each

Why is firmware extraction relevant?

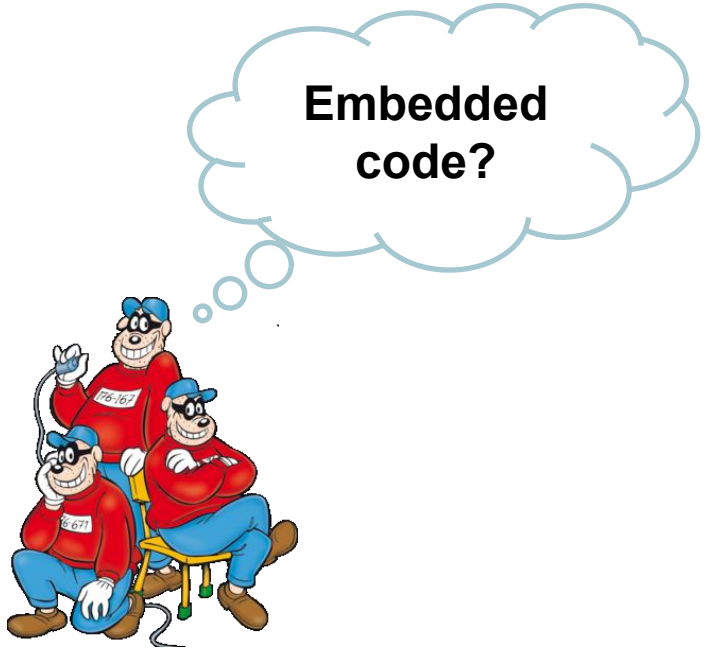
- Finding vulnerabilities
- Extraction of secret data e.g. crypto keys
- Reverse-engineering of proprietary protocols, interfaces, crypto, …
- Forensics and failure analysis
- Re-purposing of end-of-life devices
- Developing replacement firmware
- …

Durham University

Part 1: Firmware extraction from small devices – let's get invasive

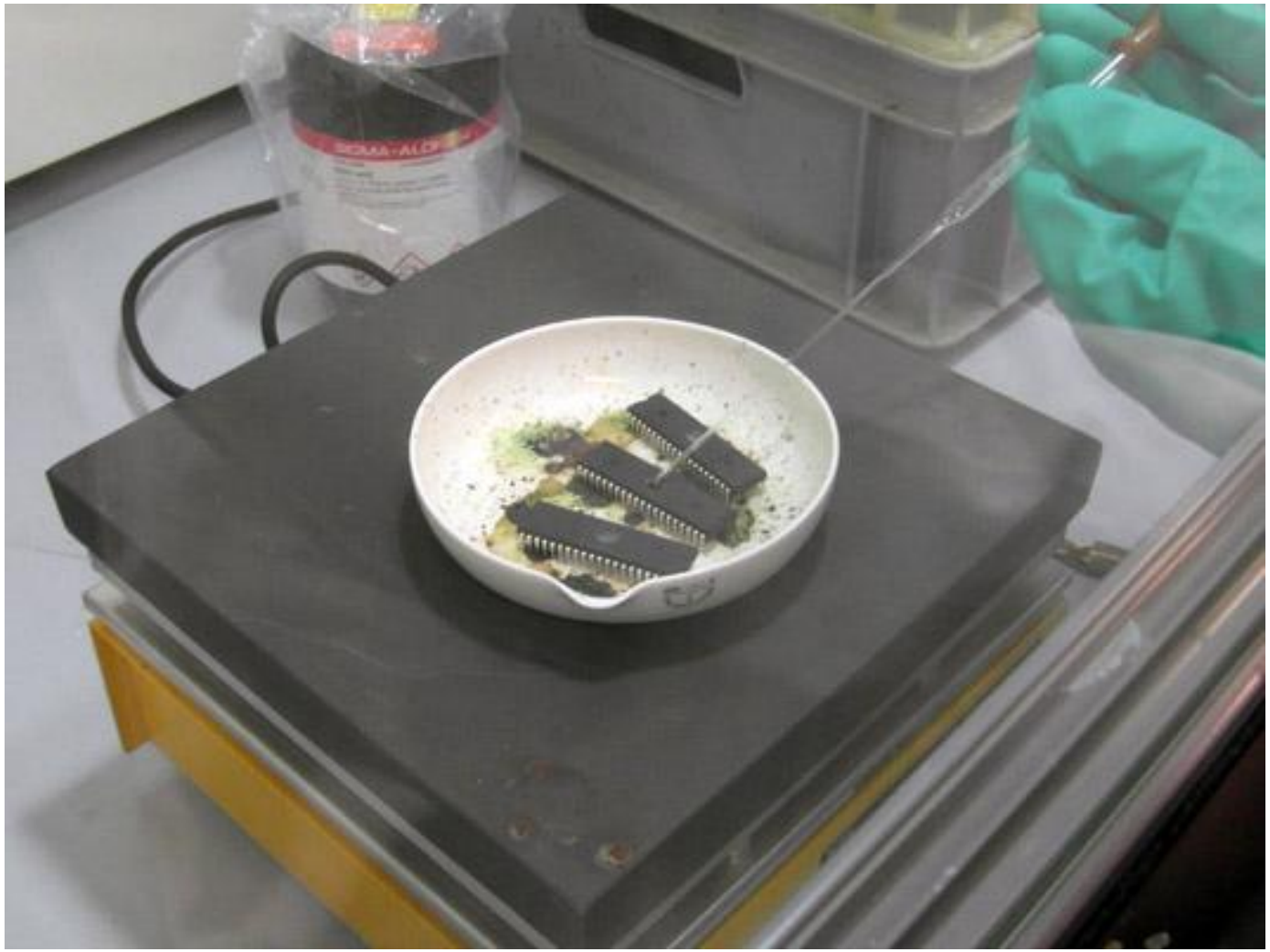
Electronic Locking System: **PIC Microcontroller in Token and Lock**





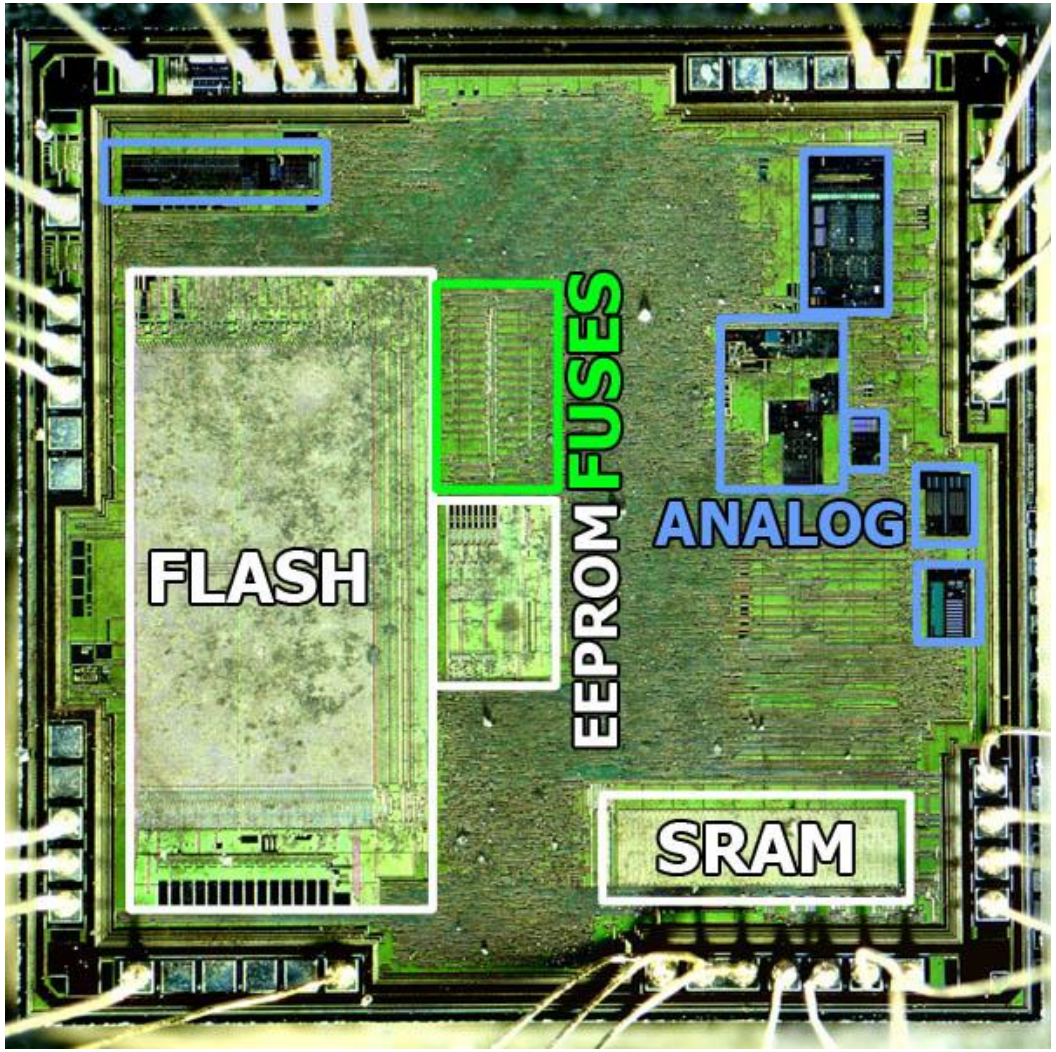
Read-out protection!

Decapping an IC (2)

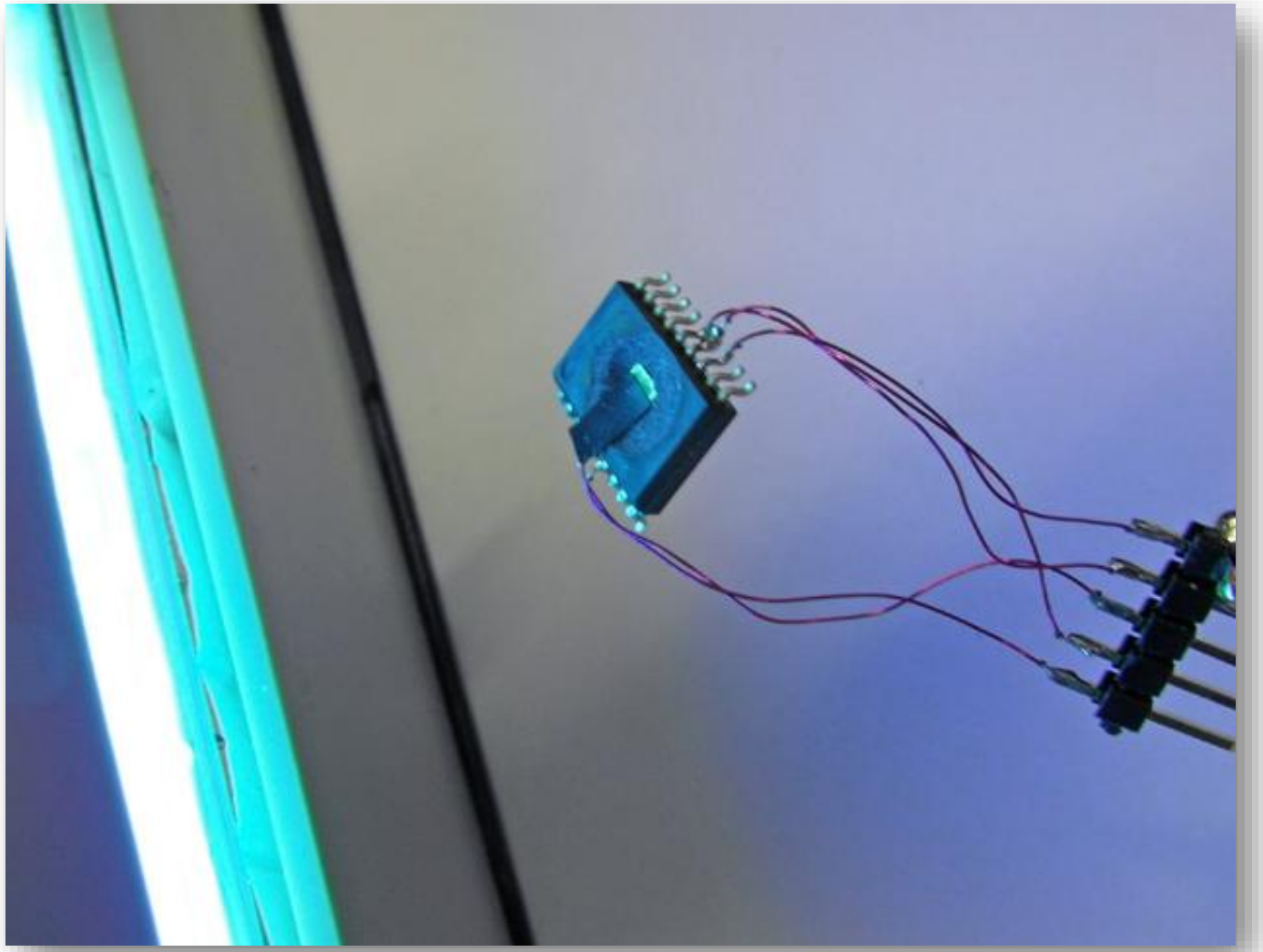


Durham University

Microscopic View of the Silicon Die

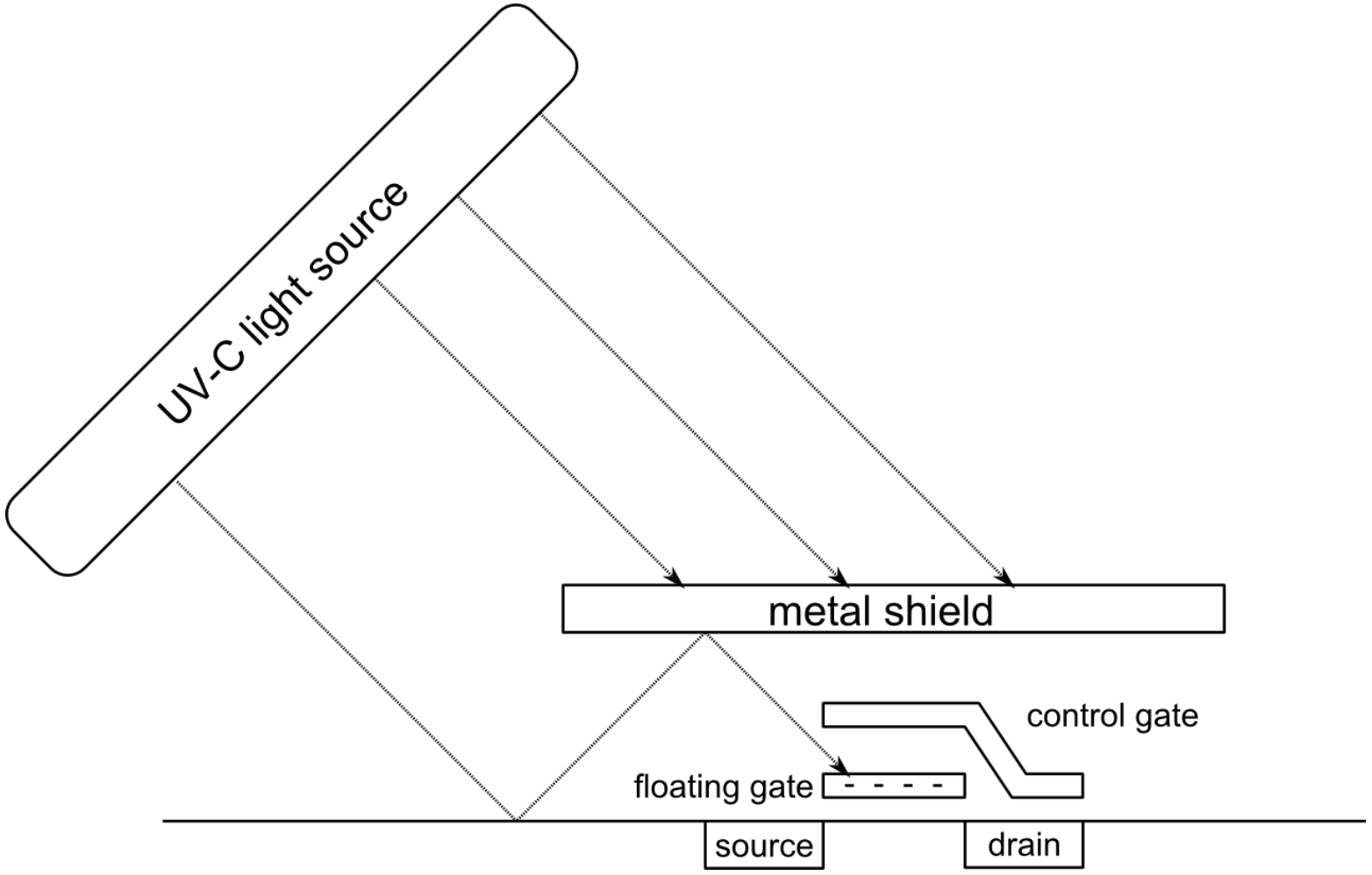


Exposure to UV-C: Disable Read-Out Protection (2)

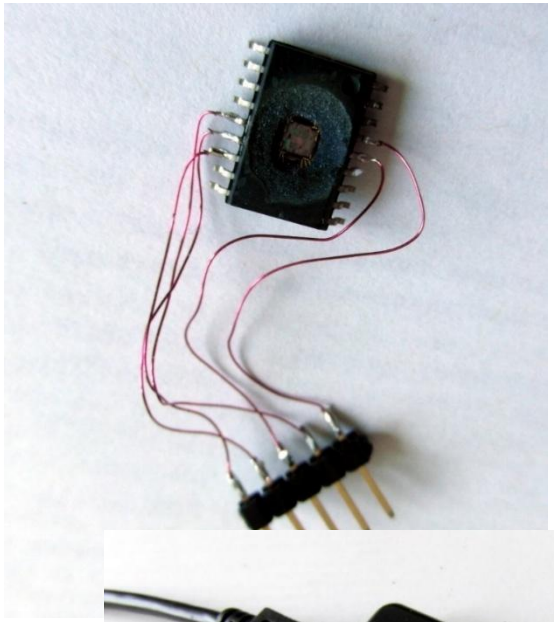


Durham University

Exposure to UV-C: Disable Read-Out Protection (2)



Read Out the Content of the PIC Microcontroller



```
CODE:0234                call    i2c_read_W_byte ; Reads W & 0x7f bytes
CODE:0234                                        ; Init I2C
CODE:0234                                        ; Set read address to Reg 71 / 70
CODE:0234                                        ; Stores (inverted) read data at location pointed to by 73 / 72
CODE:0234                                        ;
CODE:0234                                        ; Sets (75) <- W
CODE:0234                                        ; Sets bank to 0 (resets bit 5 and 6)
CODE:0235                bsf     BANK0_STATUS, 6
CODE:0236 ; assume bank = 2
CODE:0236                tstf    byte_DATA_119
CODE:0237                skpnz
CODE:0238
CODE:0238 endless_loop_1:                        ; CODE XREF: maybe_related_to_rewriting_program_memory:endless_l
CODE:0238                 b       endless_loop_1
CODE:0239                movlw   2
CODE:023A                call    read_value_from_eeprom ; Adress in W
CODE:023A                                        ; Result in W
CODE:023A                                        ; Switches bank to 0
CODE:023B                xorlw   0FF
CODE:023C                bsf     BANK2_STATUS, 6
CODE:023D                skpz
CODE:023E
```



Use standard programmer
Reverse-Engineer (e.g. IDA)
→ **all** secrets known

Durham University

Part 2: Firmware extraction from more complex devices – it gets easier…







Firmware extraction from “complex” devices

1. Check manufacturer website

Setup Video | FAQ | Firmware | Apps | GPL Code | Emulators

Firmware

A firmware update can resolve issues that the previous firmware version may have and improve its current performance.

To Upgrade

IMPORTANT: To prevent upgrade failures, please read the following before proceeding with the upgrade process

- Please upgrade firmware from the local TP-Link official website of the purchase location for your TP-Link device, otherwise it will be against the warranty. Please click here to change site if necessary.
- Please verify the hardware version of your device for the firmware version. Wrong firmware upgrade may damage your device and void the warranty. （Normally V1.x=V1）
  How to find the hardware version on a TP-Link device?
- Do NOT turn off the power during the upgrade process, as it may cause permanent damage to the product.
- Do NOT upgrade the firmware through wireless connection unless there is no LAN/Ethernet port on the TP-link device. For LTE-MiFi, it is recommended to upgrade via USB port.
- It's recommended that users stop all Internet applications on the computer, or simply disconnect Internet line from the device before the upgrade.
- Use decompression software such as WinZIP or WinRAR to extract the file you download before the upgrade.

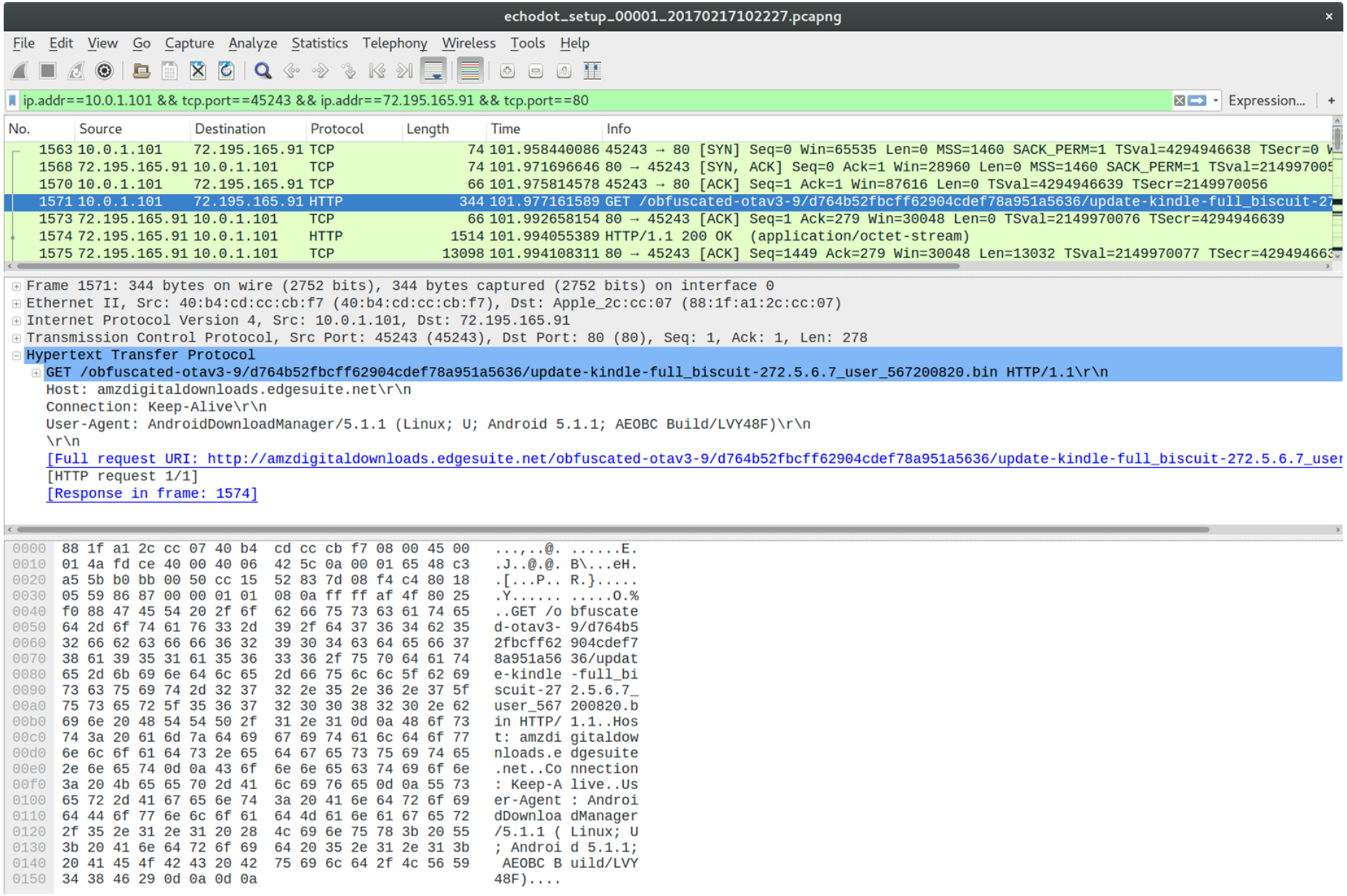
TL-WR841N(EU)_V13_180119		
Published Date: 2018-02-24	Language: English	File Size: 4.73 MB

Modification and Bug Fixes:

1. Fixed WPA2 (KRACKs) vulnerability.
2. Fixed the bug that router cannot forward some specific DNS requests.
3. Shorten the interval time to reconnect to PPPoE server after disconnection.
4. Improved the stability of Quick Setup.

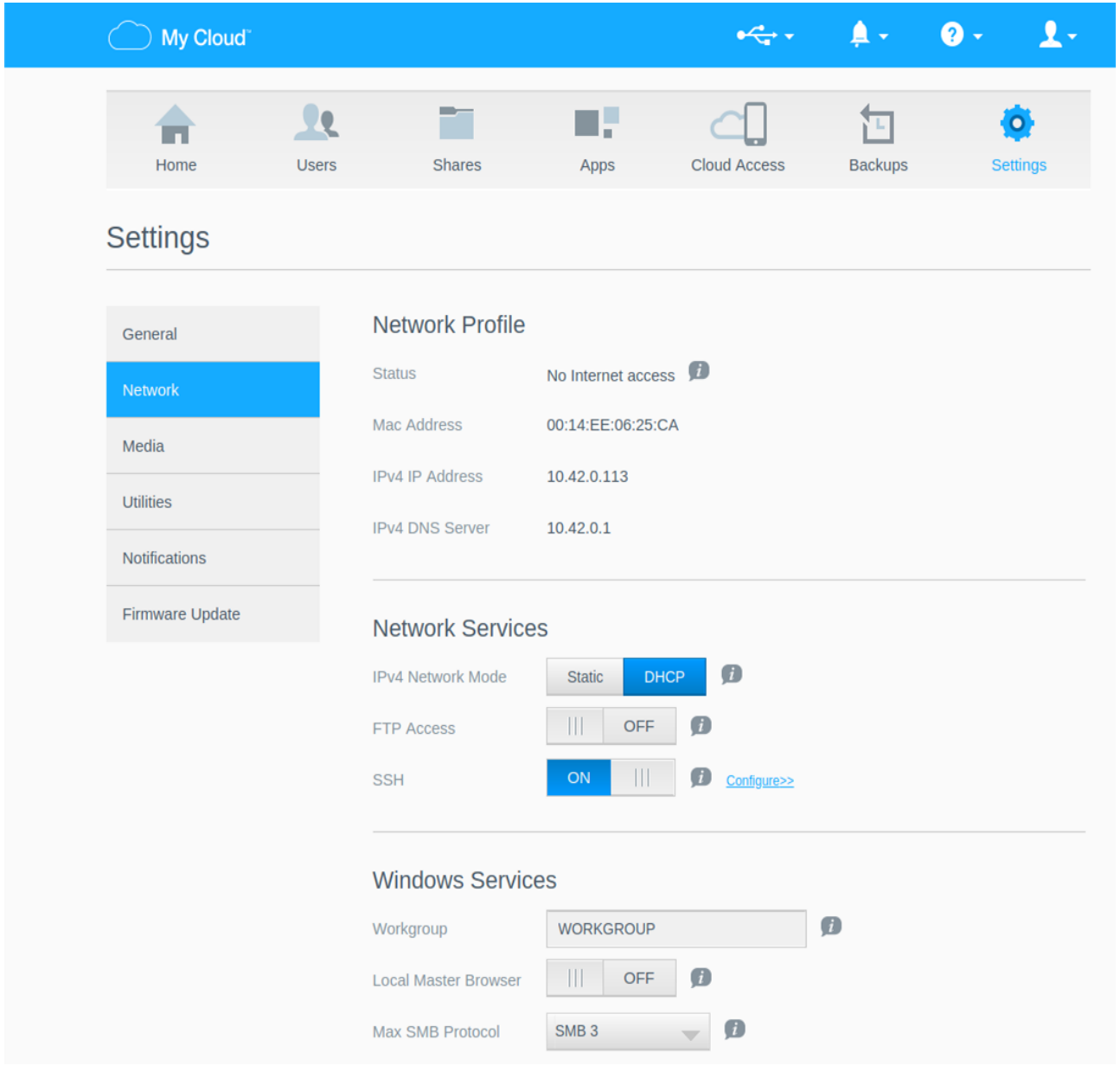
Firmware extraction from “complex” devices

1. Check manufacturer website
2. Check network traffic for firmware upgrades (plain HTTP is still used, e.g. apt-get)


echodot_setup_00001_20170217102227.pcapng
File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help
ip.addr==10.0.1.101 && tcp.port==45243 && ip.addr==72.195.165.91 && tcp.port==80
Expression...
No. Source Destination Protocol Length Time Info
1563 10.0.1.101 72.195.165.91 TCP 74 101.958440086 45243 → 80 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 SACK_PERM=1 TSval=4294946638 TSecr=0
1568 72.195.165.91 10.0.1.101 TCP 74 101.971696646 80 → 45243 [SYN, ACK] Seq=0 Ack=1 Win=28960 Len=0 MSS=1460 SACK_PERM=1 TSval=214997005
1570 10.0.1.101 72.195.165.91 TCP 66 101.975814578 45243 → 80 [ACK] Seq=1 Ack=1 Win=87616 Len=0 TSval=4294946639 TSecr=2149970056
1571 10.0.1.101 72.195.165.91 HTTP 344 101.977161589 GET /obfuscated-otav3-9/d764b52fbcff62904cdef78a951a5636/update-kindle-full_biscuit-27
1573 72.195.165.91 10.0.1.101 TCP 66 101.992658154 80 → 45243 [ACK] Seq=1 Ack=279 Win=30048 Len=0 TSval=2149970076 TSecr=4294946639
1574 72.195.165.91 10.0.1.101 HTTP 1514 101.994055389 HTTP/1.1 200 OK (application/octet-stream)
1575 72.195.165.91 10.0.1.101 TCP 13098 101.994108311 80 → 45243 [ACK] Seq=1449 Ack=279 Win=30048 Len=13032 TSval=2149970077 TSecr=429494663
Frame 1571: 344 bytes on wire (2752 bits), 344 bytes captured (2752 bits) on interface 0
Ethernet II, Src: 40:b4:cd:cc:cb:f7 (40:b4:cd:cc:cb:f7), Dst: Apple_2c:cc:07 (88:1f:a1:2c:cc:07)
Internet Protocol Version 4, Src: 10.0.1.101, Dst: 72.195.165.91
Transmission Control Protocol, Src Port: 45243 (45243), Dst Port: 80 (80), Seq: 1, Ack: 1, Len: 278
Hypertext Transfer Protocol
GET /obfuscated-otav3-9/d764b52fbcff62904cdef78a951a5636/update-kindle-full_biscuit-272.5.6.7_user_567200820.bin HTTP/1.1\r\n
Host: amzdigitaldownloads.edgesuite.net\r\n
Connection: Keep-Alive\r\n
User-Agent: AndroidDownloadManager/5.1.1 (Linux; U; Android 5.1.1; AEOBC Build/LVY48F)\r\n
\r\n
[Full request URI: http://amzdigitaldownloads.edgesuite.net/obfuscated-otav3-9/d764b52fbcff62904cdef78a951a5636/update-kindle-full_biscuit-272.5.6.7_user
[HTTP request 1/1]
[Response in frame: 1574]
0000 88 1f a1 2c cc 07 40 b4 cd cc cb f7 08 00 45 00 ...,..@. ......E.
0010 01 4a fd ce 40 00 40 06 42 5c 0a 00 01 65 48 c3 .J..@.@. B\...eH.
0020 a5 5b b0 bb 00 50 cc 15 52 83 7d 08 f4 c4 80 18 .[...P.. R.}.....
0030 05 59 86 87 00 00 01 01 08 0a ff ff af 4f 80 25 .Y...... .....O.%
0040 f0 88 47 45 54 20 2f 6f 62 66 75 73 63 61 74 65 ..GET /o bfuscate
0050 64 2d 6f 74 61 76 33 2d 39 2f 64 37 36 34 62 35 d-otav3- 9/d764b5
0060 32 66 62 63 66 66 36 32 39 30 34 63 64 65 66 37 2fbcff62 904cdef7
0070 38 61 39 35 31 61 35 36 33 36 2f 75 70 64 61 74 8a951a56 36/updat
0080 65 2d 6b 69 6e 64 6c 65 2d 66 75 6c 6c 5f 62 69 e-kindle -full_bi
0090 73 63 75 69 74 2d 32 37 32 2e 35 2e 36 2e 37 5f scuit-27 2.5.6.7_
00a0 75 73 65 72 5f 35 36 37 32 30 30 38 32 30 2e 62 user_567 200820.b
00b0 69 6e 20 48 54 54 50 2f 31 2e 31 0d 0a 48 6f 73 in HTTP/ 1.1..Hos
00c0 74 3a 20 61 6d 7a 64 69 67 69 74 61 6c 64 6f 77 t: amzdi gitaldow
00d0 6e 6c 6f 61 64 73 2e 65 64 67 65 73 75 69 74 65 nloads.e dgesuite
00e0 2e 6e 65 74 0d 0a 43 6f 6e 6e 65 63 74 69 6f 6e .net..Co nnection
00f0 3a 20 4b 65 65 70 2d 41 6c 69 76 65 0d 0a 55 73 : Keep-A live..Us
0100 65 72 2d 41 67 65 6e 74 3a 20 41 6e 64 72 6f 69 er-Agent : Androi
0110 64 44 6f 77 6e 6c 6f 61 64 4d 61 6e 61 67 65 72 dDownloa dManager
0120 2f 35 2e 31 2e 31 20 28 4c 69 6e 75 78 3b 20 55 /5.1.1 (Linux; U
0130 3b 20 41 6e 64 72 6f 69 64 20 35 2e 31 2e 31 3b ; Androi d 5.1.1;
0140 20 41 45 4f 42 43 20 42 75 69 6c 64 2f 4c 56 59 AEOBC B uild/LVY
0150 34 38 46 29 0d 0a 0d 0a 48F)....

Firmware extraction from “complex” devices

1. Check manufacturer website
2. Check network traffic for firmware upgrades (plain HTTP is still used, e.g. apt-get)
3. Check for running services (SSH, Telnet, FTP, …)


My Cloud
Home
Users
Shares
Apps
Cloud Access
Backups
Settings
Settings
General
Network
Media
Utilities
Notifications
Firmware Update
Network Profile
Status
No Internet access
Mac Address
00:14:EE:06:25:CA
IPv4 IP Address
10.42.0.113
IPv4 DNS Server
10.42.0.1
Network Services
IPv4 Network Mode
Static
DHCP
FTP Access
OFF
SSH
ON
Configure>>
Windows Services
Workgroup
WORKGROUP
Local Master Browser
OFF
Max SMB Protocol
SMB 3

Firmware extraction from “complex” devices

1. Check manufacturer website
2. Check network traffic for firmware upgrades (plain HTTP is still used, e.g. apt-get)
3. Check for running services (SSH, Telnet, FTP, …)
4. Go to hardware methods

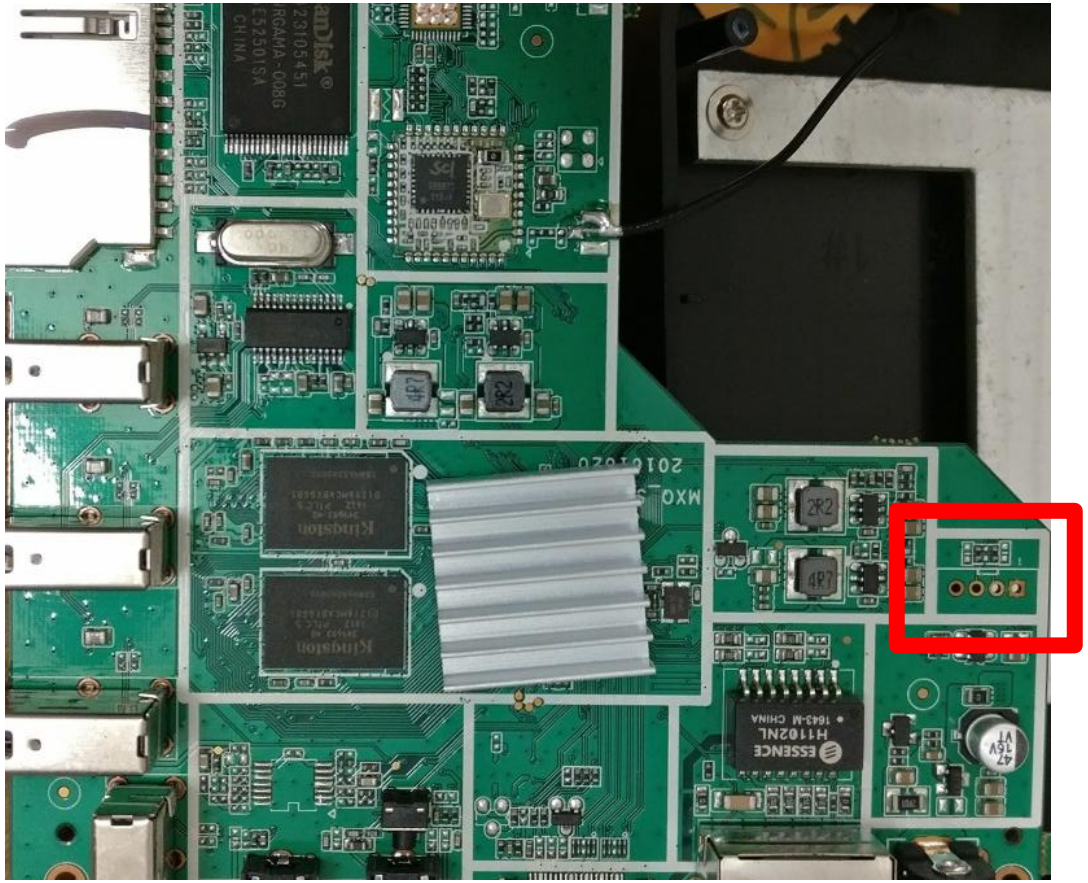
UART

- Serial communication interface from the stone age of computing but still going strong today
- Minimal pins:
  - RX = Receive
  - TX = Transmit
  - (Ground)
  - (Supply voltage)



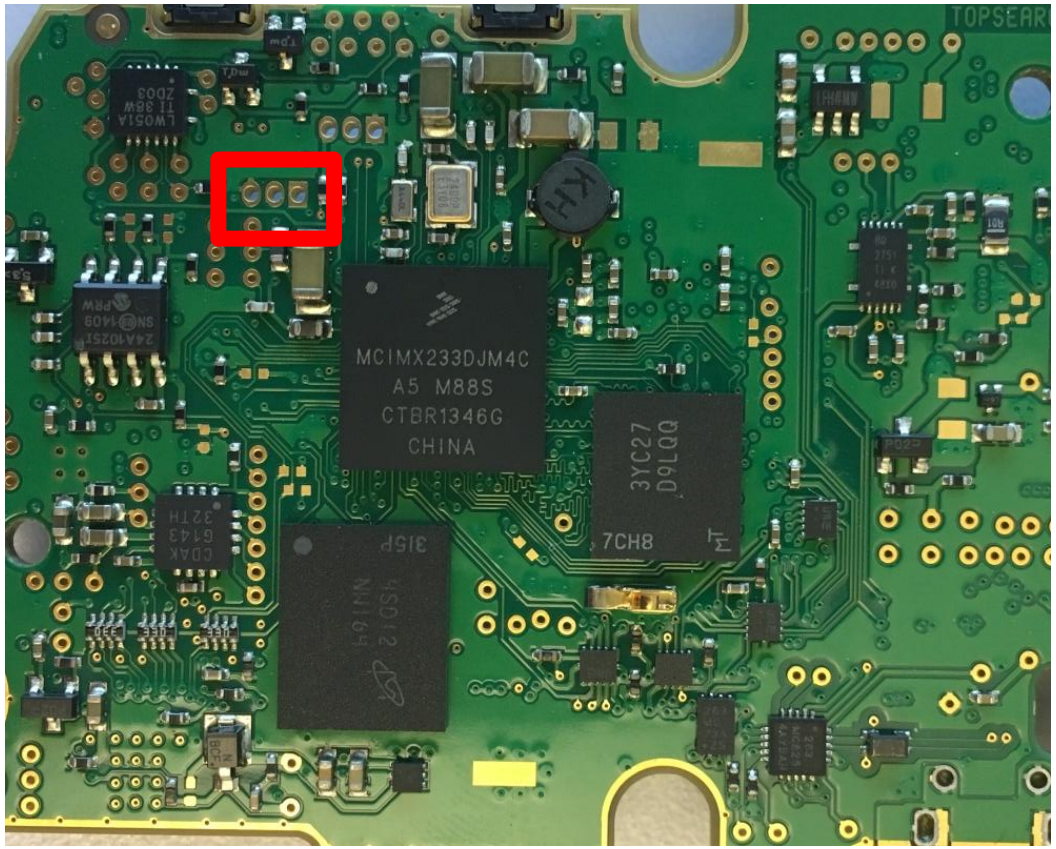


Spot the UART



Durham University

Spot the UART



UART firmware extraction

- Get access via rootshell & dump – user & pw often trivial (e.g. infotainment ECU `root/root`)
- Use the boot loader (often U-Boot)
  - Interrupt boot process
  - Enter bootloader shell
  - U-boot: `nand dump 0x0` etc.
- Alternative: start own bootloader via MCU low-level loader (e.g. on AM335x), then dump from there
  (look for `CCCCC` …)


UART
SMART HOME
PUSH NOTIFICATIONS
PHONE CONTROLLABLE
BACK UP BATTERY
USB PORTS FOR CHARGING DEVICES
EASY SETUP
WIRELESS CONNECTIVITY
EXPANDABLE

Durham University

Live-Demo

"Anything that can go wrong, will go wrong"

Extracting firmware via UART

The Swann smart home hub

- **Identify**: From boot log:
  TI AM335X ES2.1 ARMv7 CPU

- **Structure/packing:** e.g. via Binwalk …



The Swann smart home hub

- **Identify**: From boot log:
  TI AM335X ES2.1 ARMv7 CPU

- **Structure/packing:** e.g. via Binwalk …

  – Linux kernel image
  – Actual files seem to be UBIFS – not easily mounted in Autopsy or similar
  – Root hash (with strings):
  tbKdyTU4E5hZc → sjcwoz3r



blog.mgrgroup.eu › swann-firmware-update-failed ▾

Swann firmware update failed - mgr import

... AM3352BZCZ60 TI AM335x chip; 128M NAND; ZigBee chip CC2538NF23; Username root, password has is **tbKdyTU4E5hZc** which is DES for sjcwoz3r in my ...

Durham University

Live-Demo

"Anything that can go wrong, will go wrong"

Swann root shell

Durham University

Part 3: Firmware extraction exploiting embedded bootloaders

Other methods?



- Sometimes microcontrollers are left unprotected, i.e. readout protection is off, or don't have protection at all

- There can be software / logic bugs in the readout protection logic, e.g. STM32F0 series https://www.usenix.org/system/files/conference/woot17/woot17-paper-obermaier.pdf

- **Embedded bootloaders** have vulnerabilities
  – Fault attacks (more on that in a few lectures)
  – Software vulnerabilities…

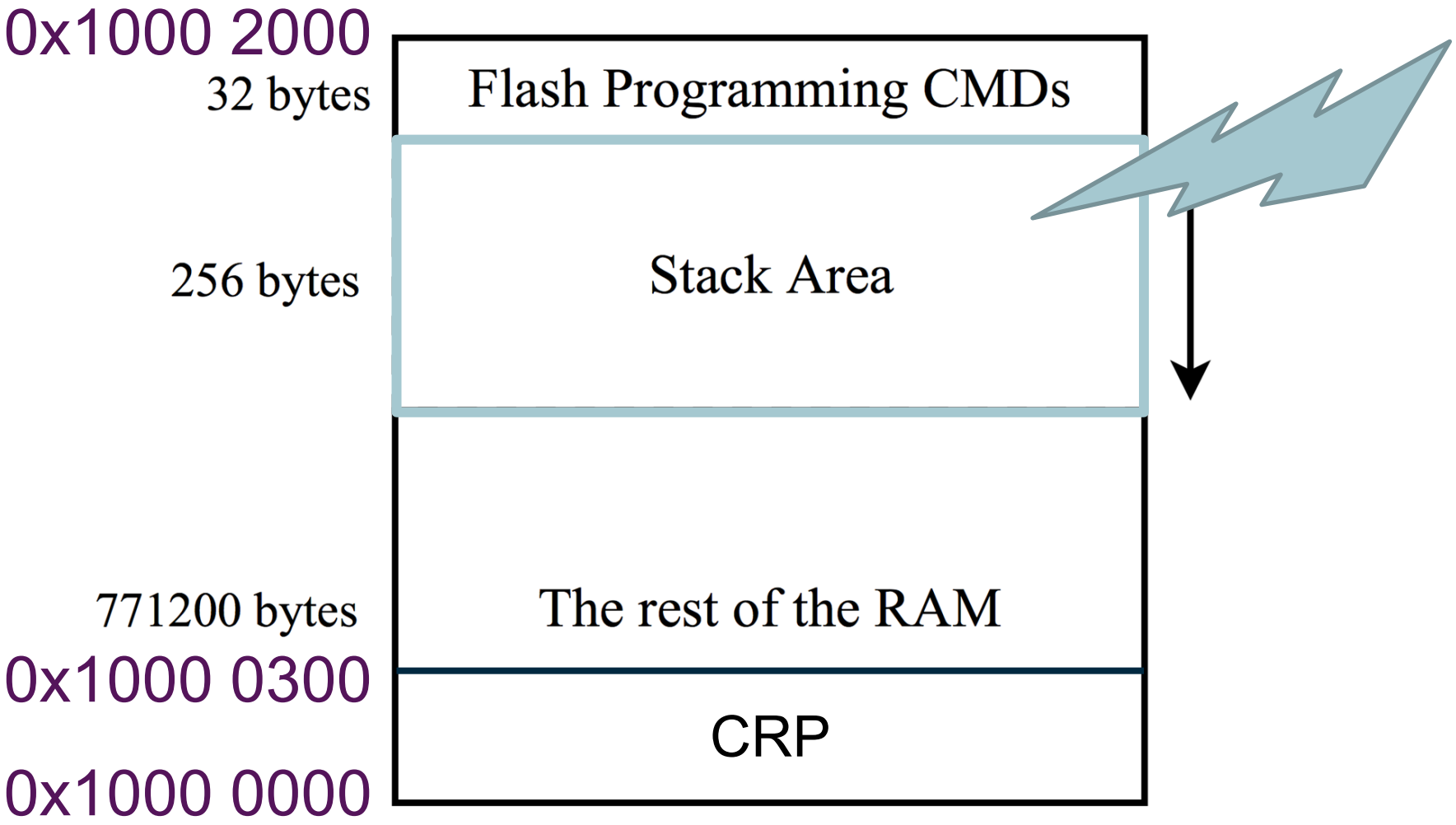
NXP ARM: LPC1343 Code Readout Protection (CRP)

ISP command	CRP1		
Unlock	ye		
Set Baud Rate	yes		
Echo	yes		
Write to RAM	yes; above 0x1000 0300 only		
Read Memory	no		
Prepare sector(s) for write operation	es		
Copy RAM to flash	s; not to sector 0		
Go	o		
Erase sector(s)	yes; sector 0 can only be erased when all sectors are erased.		
Blank check (s)	no	no	n/a
Read Part	yes	yes	n/a
Read Boot code version	yes	yes	n/a
Compare	no	no	n/a
ReadUID	yes	yes	n/a

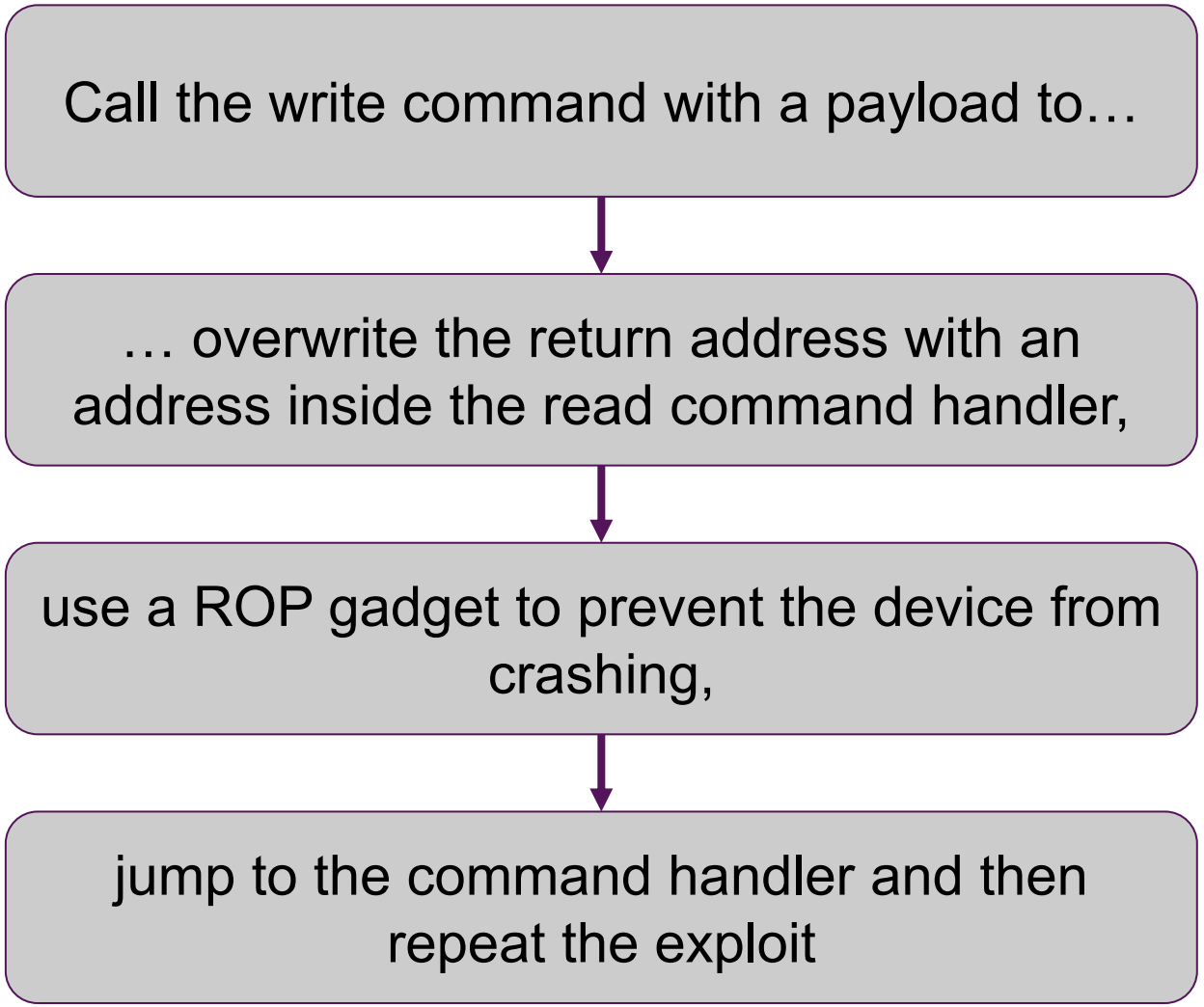
```
CRP_Check:
ldr         r0, =0x4003c000 ; dword_1fff0f8c, CODE XREF=ISP_W(write)_Command+22, ISP_W(wri
ldr         r2, [r0]
movs        r1, #0x40
orrs        r2, r1
str         r2, [r0]
ldr         r2, =0x10000184 ; dword_1fff0f90
ldr         r3, =CRP_1    ; CRP_1,dword_1fff0f94
ldr         r2, [r2]
ldr         r3, [r3]      ; CRP_1
cmp         r2, r3
Jump_if_CRP_Off:
bne         loc_1fff0da6
```

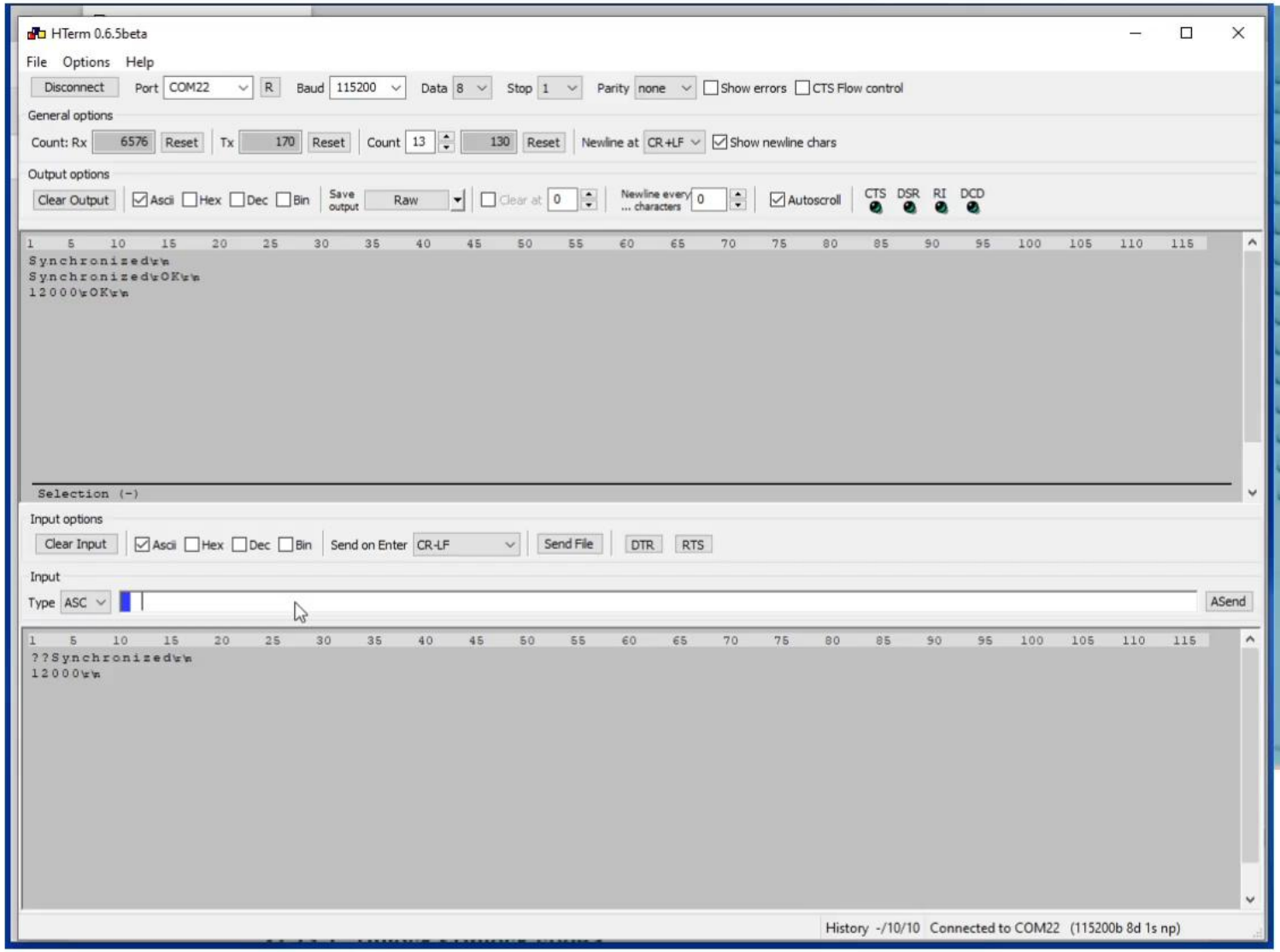
```
Address_checking(Writing_below_0x10000300_not_allowed(if_CRP_enabled):
ldr         r2, =0x438    ; dword_1fff0f98
ldr         r3, [sp, #0x28 + var_18]
ldr         r2, [r2]
adds        r2, #0xff
adds        r2, #0xff
adds        r2, #0x2
cmp         r3, r2
Jump_if_Address_Above_0x10000300:
bhs         loc_1fff0da6
```

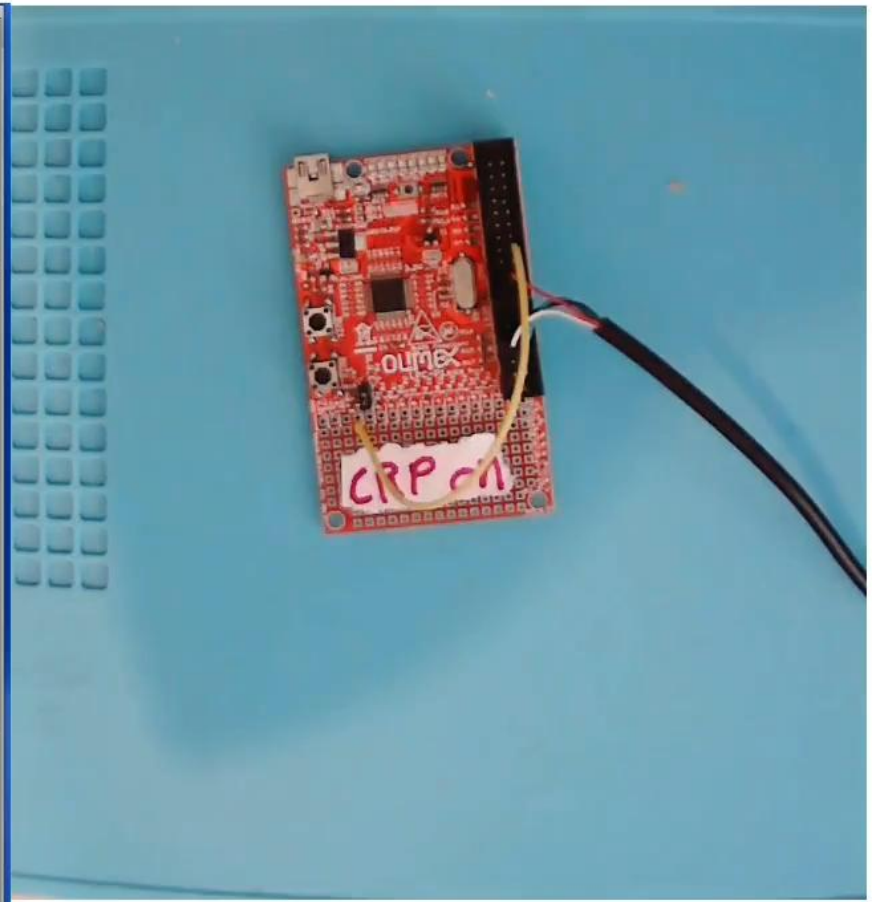
LPC1343 memory layout



Building a ROP exploit




HTerm 0.6.5beta
File Options Help
Disconnect Port COM22 R Baud 115200 Data 8 Stop 1 Parity none Show errors CTS Flow control
General options
Count: Rx 6576 Reset Tx 170 Reset Count 13 130 Reset Newline at CR+LF Show newline chars
Output options
Clear Output Ascii Hex Dec Bin Save output Raw Clear at 0 Newline every ... characters 0 Autoscroll CTS DSR RI DCD
Synchronized\r\n
Synchronized\rOK\r\n
12000\rOK\r\n
Selection (-)
Input options
Clear Input Ascii Hex Dec Bin Send on Enter CR-LF Send File DTR RTS
Input
Type ASC ASend
??Synchronized\r\n
12000\r\n
History -/10/10 Connected to COM22 (115200b 8d 1s np)


OLIMEX

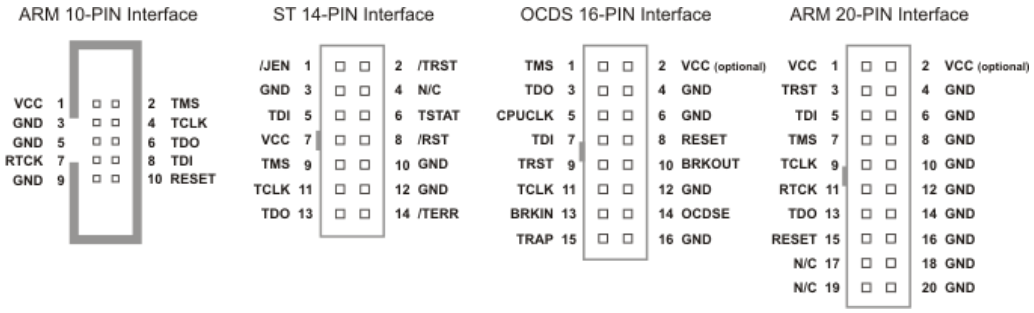
JTAG (and other test interfaces)

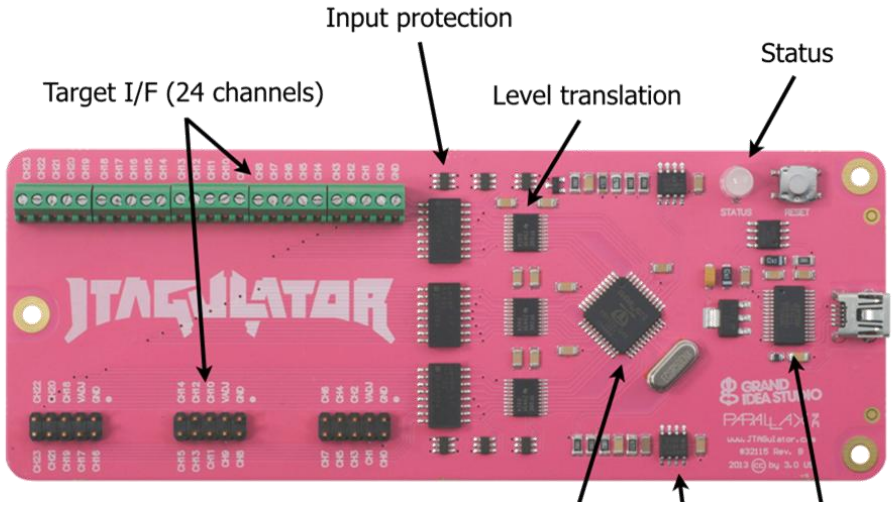
- Processors and flash chips can support a range of programming interfaces, for example:

  – JTAG (named after the Joint Test Action Group)
  – ARM SWD (Serial Wire Debug)
  – various more obscure ones

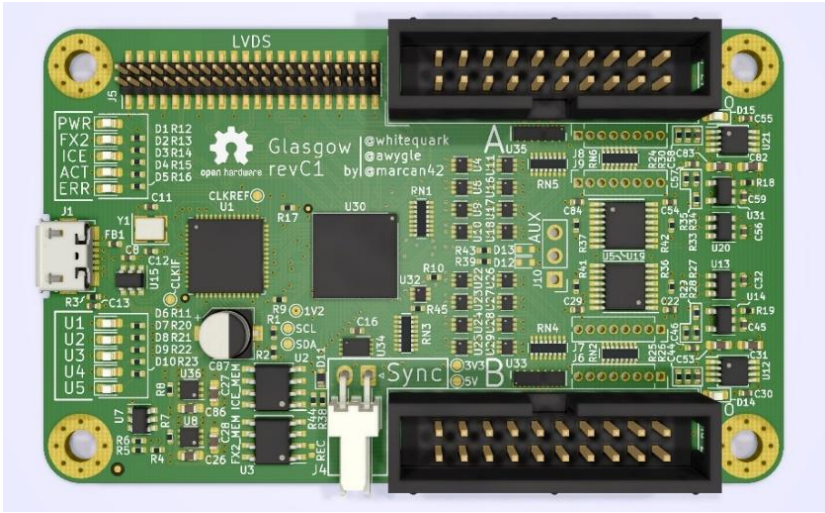
- Like UART, pin headers are often present and the interface left active

How to find JTAG pins automatically





https://github.com/grandideastudio/jtagulator/wiki



https://github.com/GlasgowEmbedded/glasgow

Reading flash memories and “chip off”

- Firmware is stored in external flash chips



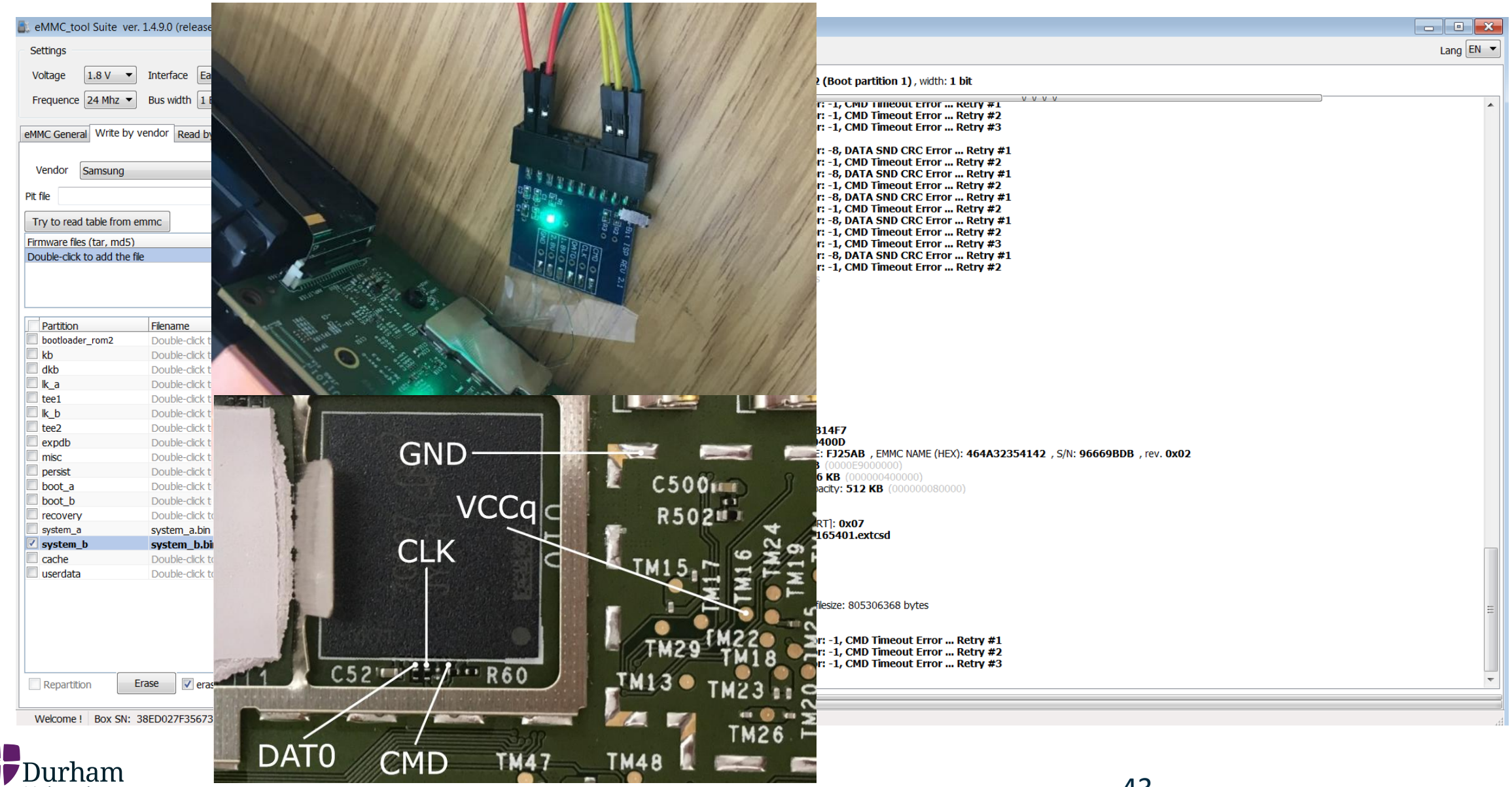






- … which can be read out (either on the board of after desoldering)

Direct eMMC access



What did we see today?

2025 Most Important Hardware Weaknesses

Top HW Weaknesses Home | Share via: X | Key Insights | Methodology | Use Cases | White Paper

Main MIHW Table

CWE ID	Name
CWE-226	Sensitive Information in Resource Not Removed Before Reuse
CWE-1189	Improper Isolation of Shared Resources on System-on-a-Chip (SoC)
CWE-1191	On-Chip Debug and Test Interface With Improper Access Control
CWE-1234	Hardware Internal or Debug Modes Allow Override of Locks
CWE-1247	Improper Protection Against Voltage and Clock Glitches
CWE-1256	Improper Restriction of Software Interfaces to Hardware Features
CWE-1260	Improper Handling of Overlap Between Protected Memory Ranges
CWE-1262	Improper Access Control for Register Interface
CWE-1300	Improper Protection of Physical Side Channels
CWE-1421	Exposure of Sensitive Information in Shared Microarchitectural Structures during Transient Execution
CWE-1423	Exposure of Sensitive Information caused by Shared Microarchitectural Predictor State that Influences Transient Execution

Expert Insights: Weaknesses Beyond Data Trends

CWE-1231	Improper Prevention of Lock Bit Modification
CWE-1233	Security-Sensitive Hardware Controls with Missing Lock Bit Protection
CWE-1244	Internal Asset Exposed to Unsafe Debug Access Level or State
CWE-1272	Sensitive Information Uncleared Before Debug/Power State Transition
CWE-1431	Driving Intermediate Cryptographic State/Results to Hardware Module Outputs

https://cwe.mitre.org/topHW/archive/2025/2025_CWE_MIHW.html

Durham University

Take-home points

- Firmware extraction is essential for pentesting and security analysis
- It is not an exact science, but rather has various “best practices”
- More complex device firmware with OS etc. are often easier to extract and analyse
- LLMs can speed up the process
- **Never** rely on firmware secrecy to protect your products (regulations such as the cyber resilience act (CRA) can mandate this)!

Durham University

Thanks! Questions & discussion